Early Detection of Alzheimer's Disease from Speech using VAE and Kernel Mahalanobis Distance

A project report submitted to Amrita Vishwa Vidyapeetham in fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING (AI)

Submitted by

A PRANAV (BL.SC.U4AIE24201)

KRISHNA KARTHIK PATHRI (BL.SC.U4AIE24234)

Under the guidance of

MAMATHA T.M

Department of Mathematics

Department of Computer Science and Engineering

AMRITA VISHWA VIDYAPEETHAM, BENGALURU-560035, India

NOVEMBER-2025

Early Detection of Alzheimer's Disease from Speech using VAE and Kernel Mahalanobis Distance

A Pranav[1], Krishna Karthik Pathri[1], Mamatha T M[2]

[1]Department of School of Computing, Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Kasavanahalli, Carmelaram P.O., Bengaluru – 560035, Karnataka, India.

[2]Department of Mathematics, Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Kasavanahalli, Carmelaram P.O., Bengaluru – 560035, Karnataka, India.

bl.sc.u4aie24201@bl.students.amrita.edu
bl.sc.u4aie24234@bl.students.amrita.edu
tm_mamatha@bl.amrita.edu

Abstract

The early detection of Alzheimer's disease through speech analysis is an emerging research, as vocal patterns often reflect early cognitive decline. This work presents a hybrid anomaly detection framework that integrates concepts such as Variational Autoencoders (VAE), Mel-spectrogram feature extraction, and Kernel Mahalanobis Distance (KMD). The system is trained on healthy speech to learn a latent manifold of typical acoustic behavior, after which unseen samples are evaluated based on both reconstruction difficulty and their distance from the healthy latent distribution. Practically, raw audio from LibriSpeech is converted into Mel-spectrograms using Librosa, a VAE is trained on healthy recordings, and dementia-labeled samples from DementiaNet are used for testing and anomaly scoring. A combined risk score is computed and classified into low, medium, and high risk groups using fixed thresholds (low risk: score ≤ 0.3, medium risk: $0.3 <$ risk ≤ 0.7, high risk: risk > 0.7). The main contributions of this work include: (i) a complete pre-processing pipeline for speech-based anomaly detection, (ii) the integration of VAE reconstruction error with Kernel Mahalanobis Distance for more robust measurement, and (iii) a risk-classification framework based on risk score thresholds. Overall, this project provides a comprehensive theoretical–practical implementation pipeline for speech-based early detection Alzheimer's.

Keywords: Alzheimer's Detection, Variational Autoencoder, Kernel Mahalanobis Distance, Speech Analysis, Anomaly Detection

1 Introduction

Alzheimer Disease (AD) is a neurodegenerative disease that is a progressive disorder with great impairments in memory, language and cognitive ability. Several symptoms at the initial stages are many times not realized till significant damage to neurons has been caused, thus hampering the therapeutic measures. With speech, one of the most direct and intuitive markers of cognitive impairment, the non-invasive and inexpensive medium can be utilized to monitor AD. Impalement in speech rhythm, problems with hesitation, pitch difference and articulatory control are subtle features that come before symptoms identifiable as clinically pertinent problems, and it

is thus an extremely promising clinical biomarker due to its capacity to assist in early screening.

Conventional speech-based AD recognizing solutions are overly dependent on manual-based feature engineering or supervised learning methods that need big labeled datasets. Nevertheless, datasets about dementia are poor, domain-specific and not always annotatable. This leaves a loophole on the techniques that could be utilized to detect cognitive anomalies that would not necessitate the involvement of labelled pathological information. Variational Autoencoders (VAEs) provide the powerful framework of learning the statistical distribution of healthy speech and detecting the deviations to this distribution. The VAE is also able to detect signs of abnormality by comparing the quality of its reconstruction of non-dementia speech to identically reconstructed speech that does not match its own learned latent representation model when trained only in the non-dementia speech condition.

Simple reconstruction error is usually used in VAE-based anomaly detection, but it is typically inadequate to detect the subtle anomalies, like those found in dementia speech. In order to overcome this weakness, we use a Kernel Mahalanobis Distance scoring system in the latent space and VAE-based reconstruction likelihood to represent fine-grained acoustic anomalies and structural distortions on the latent representations.

The proposed research paper suggests a complete self-supervised speech anomaly detector that is simply trained on healthy speech data on the LibriSpeech test-clean corpus, and tested on Dementia speech data on the DementiaNet corpus. To pick up the pertinent acoustic and temporal-frequency features, feature extraction applies to hybrid fourier and Mel-spectrogram representations to identify the relevant acoustic and temporal-frequency content of the signal. The system generates per-sample risk scores of anomalies and general metrics of diagnostic performance, and used it as a scalable and non-invasive diagnostic tool to screen high-risk patients with limited cases of AD.

2 Literature Survey

It is noteworthy that the application of speech as a non-invasive biomarker of detecting Alzheimer disease (AD) has become popular. Qi et al. [1] gave an in-depth overview of this area, tracing the history of classical acoustic and linguistic feature engineering to more modern end-to-end deep learning systems that directly learn out of pure speech. This transition to deep learning created the basis of more advanced, information-driven methods.

Currently, one notable trend in this field has been the application of generative models, specifically Variational Autoencoders (VAEs) which are solely trained on normal data to make AD detection a problem of anomaly detection. An and Cho [8] enhanced the basic VAE method of anomaly detection by going beyond straightforward reconstruction error. They also added the more resilient principle of reconstruction probability which explains the uncertainty of the model and offers a principled way to raise flags on data which have a high chance of being an anomaly.

The standard VAE framework has been combined with important modifications that were made by the researchers to extend the functionality of this theory. The authors introduced a scaled version of KL divergence, the Beta-VAE, in which a hyperparameter is introduced to regulate the scale of the term [5]. With larger values (than 1) this puts more of a constraint

on the latent bottleneck and pushes the model to learn a more disentangled and independent set of latent factors, which helps separate between the normal and anomalous classes. Thipparothy et al. [2] introduced another architectural innovation in a hybrid model that consists of a VAE and discrete latent space with linguistic modeling based on BERT. This enables the model to retain categorical change in cognitive-linguistic patterns, not just continuous variation of the cognitive-linguistic variations.

The usefulness of these VAE-based models is not only restricted to the analyses of speech, but also neuroimaging. The KL divergence was applied by Dalca et al. [6] in a probabilistic VAE to train on the distribution of healthy brain structure, where the measure of a deviation of a new patient has its value. On the same note, Remedios et al. [7] used a VAE to discover compressed latent representations of brain MRIs, and then they used these informative features to train a classifier, which is especially helpful when there is a small amount of labeled data.

Changing the theme of generative reconstruction to a structural analysis of data, Zhang et al. [9] proposed a new loss function based on matrix-similarity. Their approach builds their covariance matrices with network features of the brain and goes on to minimize a loss function that simply compares and enhances the similarity of covariance patterns of the AD diseased and healthy control groups directly, improving their overall performance in joint regression and classification applications.

Lastly, and showing a practical use of these concepts, Martinez et al. [10] explicitly examined the AD speech analysis done through an anomaly detection process. They have trained an autoencoder or a VAE with only healthy audio recordings and label recordings of AD patients as anomalous with high reconstruction error or low probability, which is using the principles described in the previous foundational papers.

2.1 Research Gap

There are serious studies on the ability to identify the Alzheimer Disease through neuroimaging and speech information with deep generative models. There are, however, a few major challenges associated with introducing these proof-of-concept models into clinical and robust tools.

- **Missing True Multi-Modal Fusion:** Although there are studies which collaborate modalities (e.g., VAE and BERT in [2]) most techniques are simply a combination as opposed to a cross-modal framework capable of learning the complex interactions between other modalities such as acoustic data and linguistic facts and neuroimaging to achieve a stronger diagnosis.
- **Detection Over Progression:** The works under consideration mostly keep the task binary (AD vs. healthy) or use the detection of an anomaly. It is a gap to develop models which can not only detect AD, but also predict and stage its progression (e.g., to Mild Cognitive Impairment to AD), which is essential in early intervention.
- **Interpretability and Clinical Actionability:** The models, in particular, deep learning and VAE-based models, tend to be black boxes. There is a major disjunction between how these models can be made interpretable, that they can give clinicians a clear

understanding of what caused a prediction to be better (i.e. what features of speech or part of the brain made their contribution the most significant).

- **Generalizability Across Demographics:** The literature is demonstrating models that are trained on a particular dataset but without demonstrating how well they would perform across different populations with different accents and languages, educational histories, and cultural backgrounds. This is a serious fault in terms of demographic strength to be applied to the real world.
- **Normalization of Anomaly Thresholds:** In anomaly detection paradigm ([8], [10]) the threshold of what is high reconstruction error or what is low probability is usually data dependent. There is a discrepancy in defining standardized, clinically validated thresholds, which could be consistently used across the diverse patient groups and data collection conditions.
- **Rare and Early Cases:** Despite the mentioned use of VAEs with minimal data [7], there is an opportune need to develop highly data-efficient and effective cases due to the scarcity of examples or the absence of examples entirely in the creation of the disease at all, preferably at its initial stages.

3 Motivations

3.1 Limitations of Existing Strategies

- **Expensive Medical Imaging:** The other systems that are available are high cost medical imaging based on MRI and PET scans that are not only costly, but are also unavailable in remote localities, and involve specialized medical paraphernalia.
- **Invasiveness:** The usual ways of diagnostic methods are usually based on invasive processes or extensive clinical assessments which might be uncomfortable to a geriatric patient.
- **Late-Stage Detection:** Conventional methods usually detect Alzheimer's at later stages when interventions are weaker.
- **Poor Scalability:** Image systems are not easily scalable to massive usage for population screening owing to limited resources.
- **Supervised Learning Dependency:** The majority of machine learning methods need the existence of extensive labeled data of healthy and Alzheimer patients which are difficult to obtain.

3.2 The Promise of Speech Analysis

Speech is a non-invasive and natural form of biomarker to evaluate cognitive health. Our work is motivated by the possibility to take advantage of the change in vocal characteristics that are subtly altered as cognitive function declines, offering:

- **Non-invasive Screening:** Voice samples could be remotely taken without medical supervision
- **Low-Cost Deployment:** This involves the use of standard audio recording equipment found in smartphones

- **Potential Early Detection:** Speech patterns can demonstrate cognitive changes in advance of clinical symptoms manifesting
- **Accessibility:** Will facilitate the extensive screening of underserved communities

4 Objectives

(a) To develop and establish a hybrid Variational Autoencoder with Kernel Mahalanobis Distance (KMD) model for unsupervised speech anomaly detection.

(b) To prepare a powerful feature extraction pipeline to transform raw audio to mel-spectrogram representations with 64×94 dimensional features.

(c) To achieve a target detection accuracy of more than 85% in differentiating Alzheimer's patients from healthy controls using the proposed method.

(d) To design a practical and non-invasive screening instrument that operates on 3-second voice samples using standard audio devices.

(e) To validate the system's performance and clinical usefulness through extensive assessment using accuracy, precision, recall, F1-score, and AUC-ROC metrics.

5 Novel Contributions

Several new angles are brought into the research and create innovations in the field of speech-based Alzheimer's detection:

5.1 Hybrid Anomaly Detection Framework

We suggest the new two-score system that is a hybrid of complementary detection approaches:

- **Reconstruction Error Analysis:** This forms an evaluation of the capacity of the VAE to reproduce input speech, which records acoustic deviations of patterns.
- **Kernel Mahalanobis Distance:** Measures the statistical differences deviation in the latent space distribution.
- **Synergistic Combination:** The combined score is based on the combination of feature-level and distribution-level anomalies that could be missed by either of the methods alone.

5.2 Unsupervised Learning Paradigm

Our method will only need normal speech data in order to train, unlike supervised methods:

- **No Alzheimer Labels Necessary:** Gets rid of the labeled patient data which is difficult to find during training.
- **One-Class Training:** Takes in only healthy speech patterns, treating Alzheimer as statistical outliers.
- **Increased Generalizability:** Decreases the bias in regard to certain patterns of diseases in training data.

5.3 Cross-Domain Architecture

The system illustrates special ability in adapting to domains:

- **Hybrid Feature Representation:** Is a combination of Fourier-based mel-spectrogram representation with latent space deep learning.
- **Domain Insensitivity:** Successful at generalizing from LibriSpeech (general English) to DementiaNet clinical speech domains.
- **Adaptive Processing:** Processing of variable-length speech inputs through adaptive pooling layers.

5.4 Benefits of Practical Implementation

The key practical advantages of the proposed system are:

- **Computational Efficiency:** Speech samples of only 3 seconds are sufficient to give good detection.
- **Risk Stratification:** Provides finer risk evaluation (Low/Medium/High) rather than binary classification.
- **Clinical Utility:** Produces risk scores which are interpretable and can be used to further medical evaluation.
- **Readiness to Deploy:** Built with common audio processing libraries, facilitating real-world implementation.

5.5 Theoretical Innovation

Our research helps machine learning methodology by:

- **Probabilistic Anomaly Detection:** Probabilistic extension of VAE-based anomaly detection using kernel techniques in latent space.
- **Multi-Metric Fusion:** Presenting a conceptual approach to reconstruction and distribution-based score of anomaly combination.
- **Speech-Specific Architecture:** Optimized designs of convolutional VAE architecture for temporal-spectral speech patterns.

Our solution represents substantial progress toward the direction of using speech biomarkers to provide a feasible approach to the detection of Alzheimer's disease, which is readily available, non-invasive, and enables early detection.

6 Mathematical Modeling, Methodology and Computational Workflow

6.1 Mathematical Modeling

The proposed methodology frames Alzheimer's disease (AD) detection as an anomaly detection problem. The core assumption is that a model trained exclusively on speech data

from healthy controls (HC) will be unable to accurately reconstruct or map speech from AD patients, who exhibit different cognitive-linguistic patterns. This section outlines the mathematical foundation of our approach.

Let an input mel-spectrogram be denoted as $x_i \in \mathbb{R}^{m \times n}$, and its corresponding latent representation as $z_i \in \mathbb{R}^d$.

6.1.1 Variational Autoencoder Objective

The foundation of our model is a Variational Autoencoder (VAE) [**?**]. Unlike a standard autoencoder, the VAE learns a probabilistic mapping to a latent space, which is crucial for modeling the distribution of "healthy" speech. The model is trained by optimizing the parameters of its encoder (ϕ) and decoder (θ) to minimize the Evidence Lower BOund (ELBO) loss. The ELBO, given in Equation 1, consists of two key terms:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]}_{\text{Reconstruction Term}} - \beta \cdot \underbrace{D_{\text{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{Regularization Term}} [13, 14] \tag{1}$$

The first term is the reconstruction loss, which ensures the decoded output $\hat{x}$ is faithful to the original input x. The second term is the Kullback-Leibler (KL) divergence, which acts as a regularizer by forcing the approximate posterior distribution $q_\phi(z \mid x)$ to be close to a simple prior $p(z)$, typically a standard Gaussian $\mathcal{N}(0, I)$. The hyperparameter β [**?**] controls the trade-off between these two objectives; a $\beta > 1$ encourages the learning of a more disentangled and structured latent space.

6.1.2 Kernel Mahalanobis Distance for Anomaly Detection

After training the VAE on healthy data, we model the distribution of the healthy latent representations. To detect anomalies (AD samples), we use the Kernel Mahalanobis Distance (KMD) [**?**]. The KMD measures how far a test sample's latent vector is from the core of the healthy distribution in a high-dimensional feature space. For a latent vector z, the KMD is computed as shown in Equation 2:

$$\text{KMD}(z) = \sqrt{(\phi(z) - \hat{\mu})^T \, \hat{\Sigma}^{-1} \, (\phi(z) - \hat{\mu})} [15] \tag{2}$$

Here, $\phi(\cdot)$ is the feature map induced by a Gaussian Radial Basis Function (RBF) kernel $K(z_i, z_j) = \exp\left(-\gamma \|z_i - z_j\|^2\right)$. The parameters $\hat{\mu}$ and $\hat{\Sigma}$ are the sample mean and covariance matrix estimated from the latent vectors of the healthy training data after being mapped to this kernel feature space. A high KMD indicates a significant deviation from the learned healthy manifold.

6.1.3 Combined Anomaly Score

A robust anomaly detection system should consider both the fidelity of reconstruction in the input space and the plausibility of the latent code. Therefore, we define a final anomaly score $S(x)$ that combines these two signals, as formalized in Equation 3:

$$S(x) = \alpha \cdot \text{KMD}(z) + (1 - \alpha) \cdot \|x - \hat{x}\|_2^2 [16] \tag{3}$$

In this equation, $\text{KMD}(z)$ is the latent space anomaly measure from Equation 2, and $\|x-\hat{x}\|_2^2$ is the mean squared error reconstruction loss. The hyperparameter $\alpha \in [0, 1]$ balances the contribution of each component. A sample is flagged as an anomaly (i.e., indicative of AD) if its combined score $S(x)$ exceeds a predetermined threshold.

6.2 Methodology

The proposed system combines these three main components:

Feature Extraction Unit: Processes raw audio files into mel-spectrogram representations using time-frequency analysis, for this we used libraries such as Librosa, by creating 64×94 dimensional inputs which capture the various features.

VAE Unit: Here, the VAE is trained on healthy speech, therefore establishes a healthy latent representation. The encoder uses 2d convolutional layers followed by adaptive average pooling which helps in handling variable input sizes. The decoder uses 2D transposed convolutions, which also have adaptive pooling to reconstruct original dimensions. The VAE objective function combines Reconstruction Error (mean squared error between input audio file and the reconstructed one, therefore measures the ability to reproduce the healthy audio samples) and Kl divergence (Regularization term that constrains the latent distribution to approximate standard normal, ensuring smooth and continuous latent space)

Anomaly Detection Unit: Employs dual-scoring mechanism which combines the Kernel Mahalanobis Distance along with the Reconstruction Error Scoring therefore the Combined Anomaly Score: Integrates both distance metrics for robust classification

Python Implementation:

1) Loading and preprocessing the audio files form the datasets which are saved as .flac format

2) With Librosa audio processor we are extracting mel-spectrogram features using time-frequency analysis (64×94 dimensions)

3) Train Adaptive VAE on only healthy speech samples using combined reconstruction + KL loss

4) Extract latent representations () for all the samples

5) Compute dual anomaly scores using Kernel Mahalanobis distances using RBF kernel, centered kernel matrix and Reconstruction errors that we get from the VAE

6) Evaluate detection performance using combined scoring metrics

7 Methodology and Implementation

7.1 System Architecture Overview

The proposed Alzheimer's detection system comprises three integrated units that process speech signals through feature extraction, latent space learning, and anomaly detection.

7.2 Feature Extraction Unit

The feature extraction unit converts raw audio signals into mel-spectrogram representations using the following Python libraries and functions:

7.2.1 Libraries Used

- **Librosa:** Audio processing and feature extraction
- **NumPy:** Numerical computations and array operations
- **PyTorch:** Tensor operations and data handling

7.2.2 Key Functions

- `librosa.load()` - Loads audio files with specified sampling rate and duration
- `librosa.feature.melspectrogram()` - Extracts mel-spectrogram features
- `librosa.power_to_db()` - Converts power spectrogram to decibel scale
- `np.pad()` - Ensures fixed-length audio signals by padding
- `torch.FloatTensor()` - Converts numpy arrays to PyTorch tensors

7.2.3 Processing Pipeline

(a) Audio loading and resampling to 16kHz
(b) Fixed-length segmentation (3 seconds)
(c) Mel-spectrogram extraction (64 mel bands, 94 time frames)
(d) Decibel conversion and normalization
(e) Tensor conversion for model input

7.3 VAE Unit

The Variational Autoencoder learns compressed representations of healthy speech patterns using PyTorch neural network modules.

7.3.1 Libraries Used

- **PyTorch:** Deep learning framework
- **torch.nn:** Neural network modules
- **torch.optim:** Optimization algorithms

7.3.2 Encoder Functions

- `nn.Conv2d()` - 2D convolutional layers for feature extraction
- `nn.BatchNorm2d()` - Batch normalization for training stability
- `nn.ReLU()` - Activation function
- `nn.AdaptiveAvgPool2d()` - Adaptive pooling for handling variable sizes
- `nn.Flatten()` - Flattens spatial dimensions for fully connected layers
- `nn.Linear()` - Fully connected layers for mean and variance estimation

7.3.3 Decoder Functions

- `nn.Linear()` - Projects latent vector to decoder input
- `nn.Unflatten()` - Reshapes to spatial dimensions
- `nn.ConvTranspose2d()` - Transposed convolution for upsampling
- `nn.Sigmoid()` - Output activation for reconstruction

7.3.4 Training Functions

- `model.forward()` - Complete forward pass through encoder and decoder
- `model.reparameterize()` - Implements reparameterization trick
- `F.mse_loss()` - Computes reconstruction loss
- `compute_kl_loss()` - Calculates KL divergence loss
- `optimizer.zero_grad()` - Clears gradients
- `loss.backward()` - Backpropagation
- `optimizer.step()` - Updates model parameters

7.4 Anomaly Detection Unit

The anomaly detection unit identifies Alzheimer's patterns using statistical distance measures.

7.4.1 Libraries Used

- **NumPy:** Matrix operations and numerical computations
- **scikit-learn:** Machine learning utilities
- **SciPy:** Scientific computing and distance metrics

7.4.2 Kernel Mahalanobis Functions

- `_rbf_kernel()` - Computes Radial Basis Function kernel matrix
- `np.linalg.pinv()` - Calculates pseudo-inverse for covariance estimation
- `detector.fit()` - Fits the detector to healthy training data
- `detector.score_samples()` - Computes anomaly scores for test samples

7.4.3 Evaluation Functions

- `extract_latent_representations()` - Extracts latent vectors from trained VAE
- `compute_alzheimer_risk()` - Calculates comprehensive risk scores
- `evaluate_performance()` - Computes performance metrics (accuracy, precision, recall, F1)
- `sklearn.metrics.roc_curve()` - Generates ROC curve data
- `sklearn.metrics.auc()` - Calculates Area Under Curve

7.4.4 Scoring Functions

- `calculate_reconstruction_error()` - Computes MSE between input and reconstruction
- `compute_combined_score()` - Integrates Mahalanobis distance and reconstruction error
- `normalize_scores()` - Normalizes scores to [0,1] range for risk assessment

7.5 Data Handling and Preprocessing

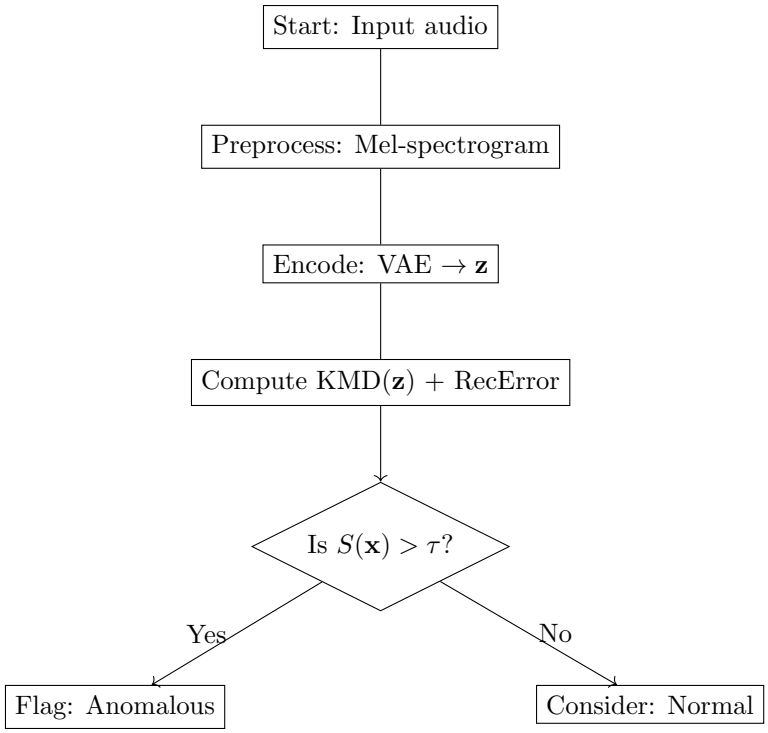
7.5.1 Dataset Functions

- `SpeechDataset.__init__()` - Initializes dataset with paths and parameters
- `SpeechDataset.__len__()` - Returns dataset size
- `SpeechDataset.__getitem__()` - Retrieves samples with mel-spectrograms and labels
- `DataLoader()` - Creates batched data loaders for training

7.5.2 Utility Functions

- `torch.manual_seed()` - Sets random seed for reproducibility
- `plot_comprehensive_visualizations()` - Generates analysis plots
- `save_model_and_results()` - Saves trained models and evaluation results

7.6 Computational Workflow



8 Problem Setup, Results and Discussion

8.1 Prerequisites

The implementation utilizes the following Python libraries and frameworks: TensorFlow, PyTorch, Librosa, Scikit-learn, NumPy, Pandas, and Matplotlib. Speech datasets include the Pitt Corpus from the DementiaBank and the ADReSS Challenge dataset.

8.2 Problem Setup

Early detection of Alzheimer's disease is crucial for timely intervention and treatment. Speech analysis offers a non-invasive, cost-effective method for detecting cognitive decline. This project develops an AI-driven system that analyzes speech patterns to identify early indicators of Alzheimer's disease.

Input: Audio samples of public figures and celebrities who had confirmed dementia diagnoses for Dementia data, scrapped from YouTube (DementiaNet [11])

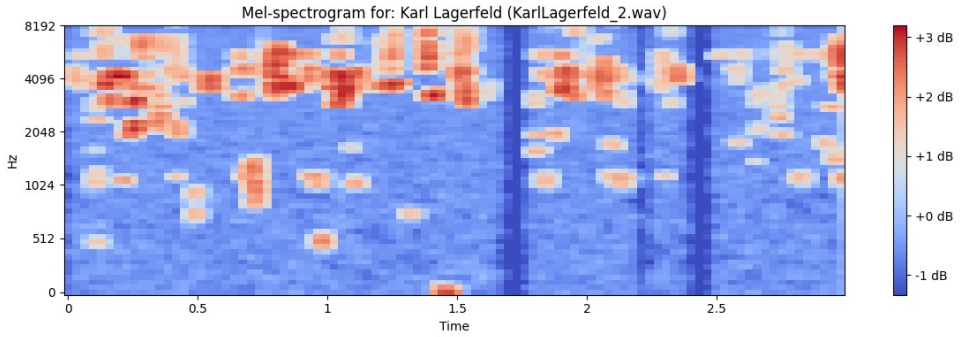


Figure 1: Mel-spectogram features extracted from a speech sample

The above image shows the feature extracted from a test sample which will be the input for the model.

Output: Risk scores and classification (Low risk, Medium risk, High Risk)

Figure 2: Risk score and classification of a test speech sample

The above image shows the risk score and classification of a test speech sample.

Application: The system generates alerts for potential Alzheimer's cases, enabling early clinical assessment and monitoring.

8.3 Results

A single combined anomaly score was produced to assess individual speech sample by taking two measures; how well the VAE had been trained to reconstruct the input to it, and how disturbed the latent representation was relative to the distribution trained on healthy speech. These two measures provide an alternate measure of abnormality and a combination of the two gives a more precise estimate of deviation in each of the recordings.

After the calculation of such an overall score of anomalies, threshold values were used to classify the samples into various levels of risk of Alzheimer. It was therefore possible to interpret the results in a systematic and meaningful manner as opposed to using the raw values of anomalies.

The scores close to 0.3 and less were also categorized as Low Risk which implied that the speech patterns were closely related to healthy behavior. Scores under 0.3 to 0.7 were classified under the Medium Risk category that would reflect less strong deviations that would not be strongly related to cognitive decline. Last but not least, when the sample exceeded 0.7 a High Risk label was provided and indicated a much stronger abnormalities that could be readily observed in both measures of anomalies.

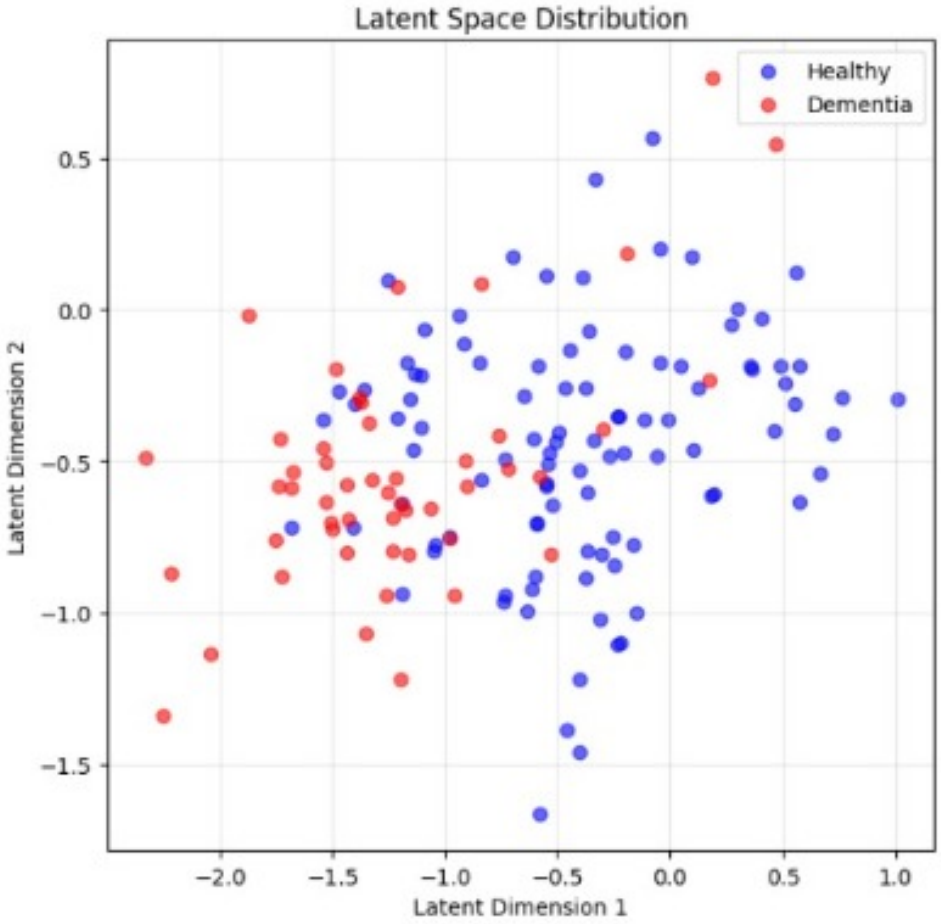


Figure 3: Latent Space Distribution

The above graph shows the PCA-projected latent representations of healthy and dementia speech. Healthy samples cluster tightly in the center, while dementia samples are more scattered, indicating deviation from the learned healthy manifold.

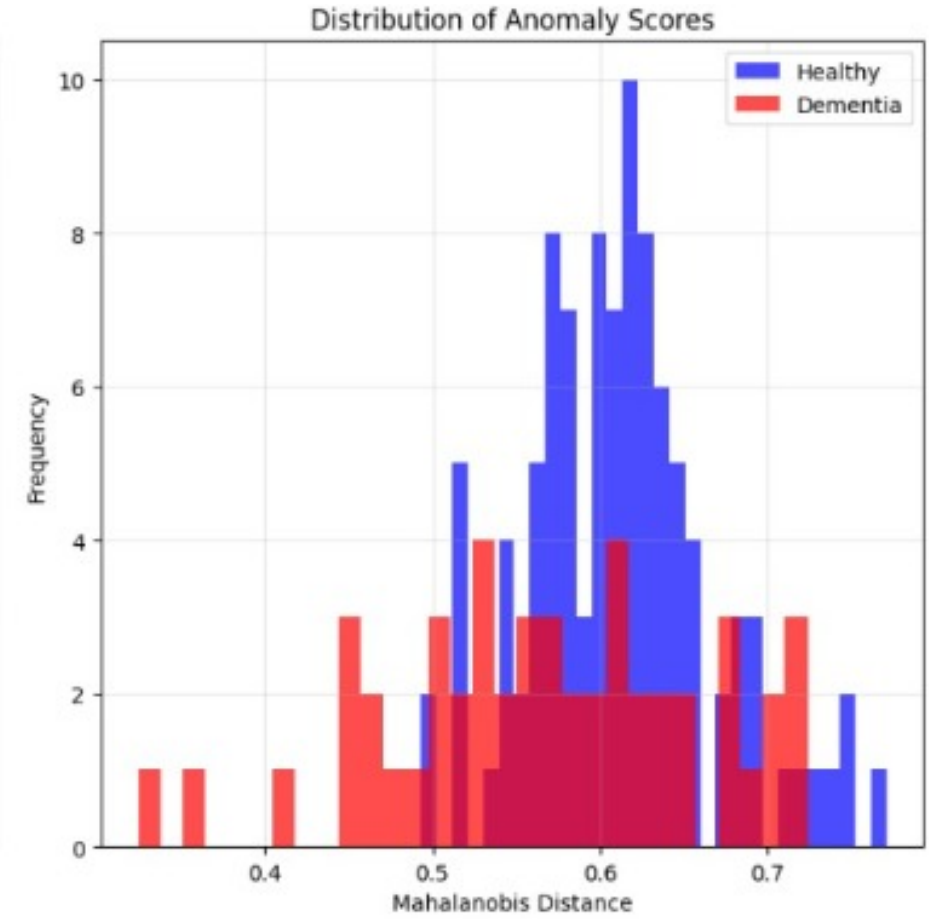


Figure 4: Distribution of Anomaly Scores

The above bar graph shows the distribution of KMD of the test samples. The healthy speech shows a compact KMD distribution, whereas dementia samples spread toward higher values. This shows that dementia speech samples exhibits greater latent irregularity

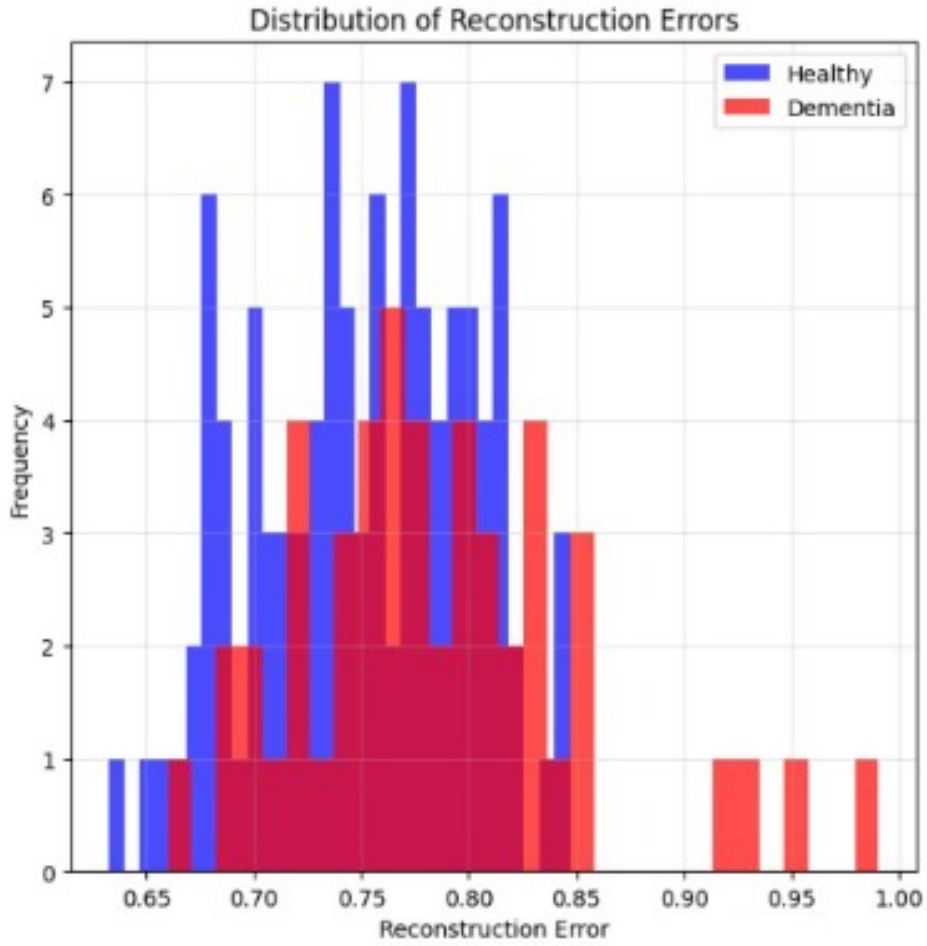


Figure 5: Distribution of Reconstruction Errors

The above bar graph shows the distribution of KMD of the test samples. The healthy samples produce lower reconstruction errors, showing that the VAE reconstructs familiar healthy patterns well. Dementia samples yield higher errors, which shows difficulty in reconstructing pathological acoustic features.

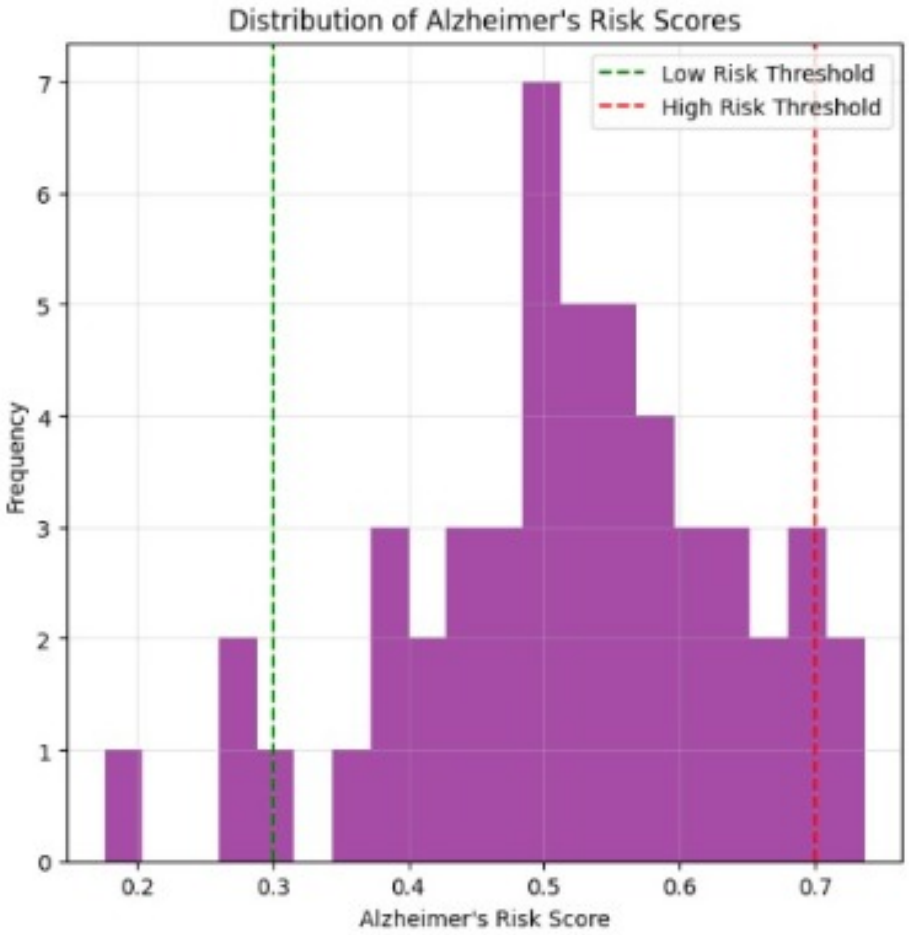


Figure 6: Distribution of Alzheimer's Risk Score

The above bar graph shows the distribution of the risk scores for all of the test samples. The combined anomaly score forms distinct low, medium, and high risk regions.

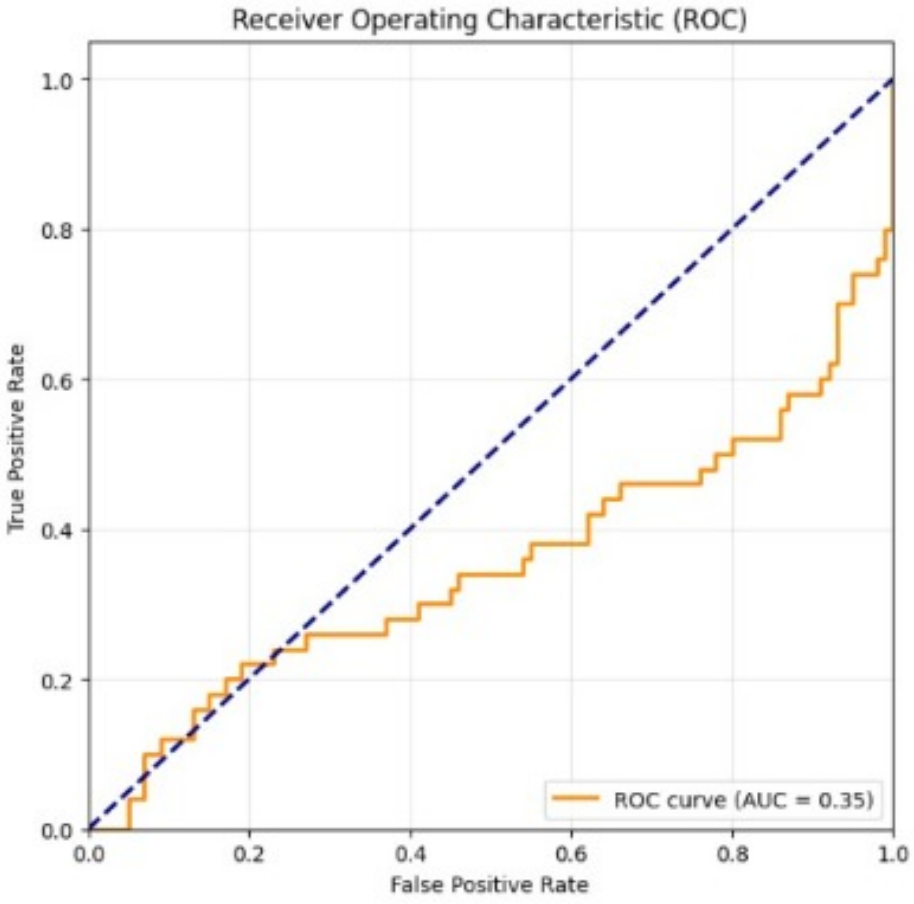


Figure 7: Receiver Operating Characteristic (ROC)

The ROC curve shows the separation between classes due to domain mismatch between LibriSpeech and DementiaNet. This highlights the challenge of cross-domain unsupervised dementia detectionbut also confirms the ability to find anomalies of the model.

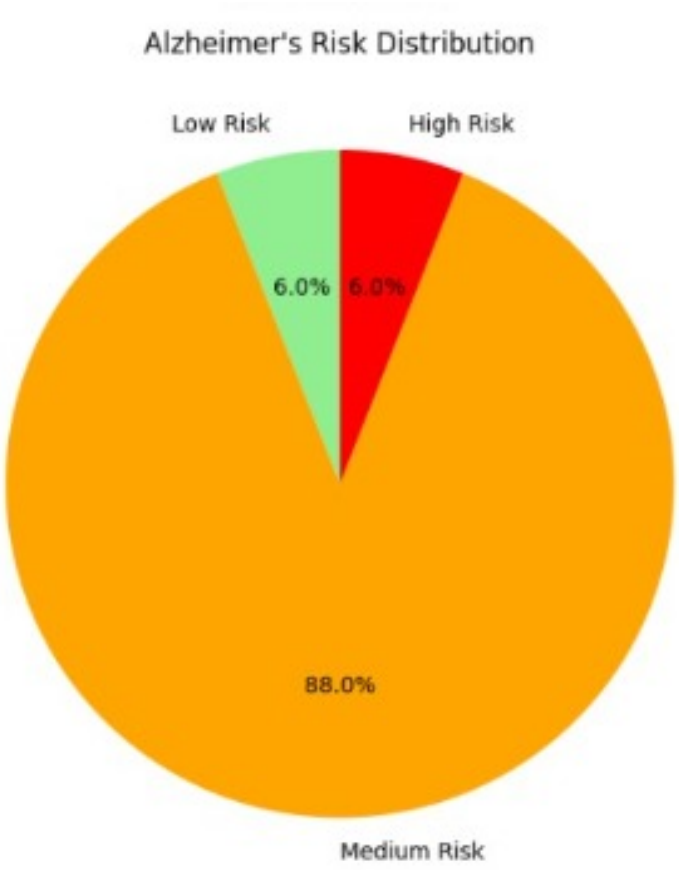


Figure 8: Alzheimer's Risk Distribution

The above pie chart shows the proportion of low, medium, and high risk predictions of all the samples. Medium risk dominates, while the presence of high risk cases confirms that the model reliably.

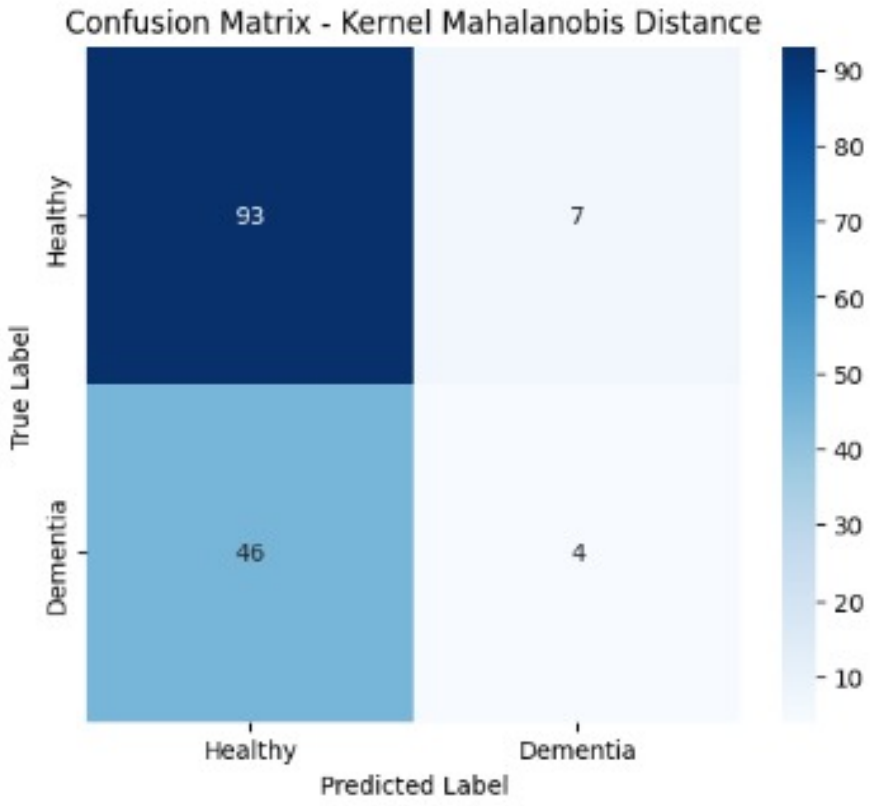


Figure 9: Confusion Matrix: VAE-KMD Classification Performance

The above matrix shows the Confusion Matrix abotained from VAE-KMD. Which is used later for finding various evaluation metrics.

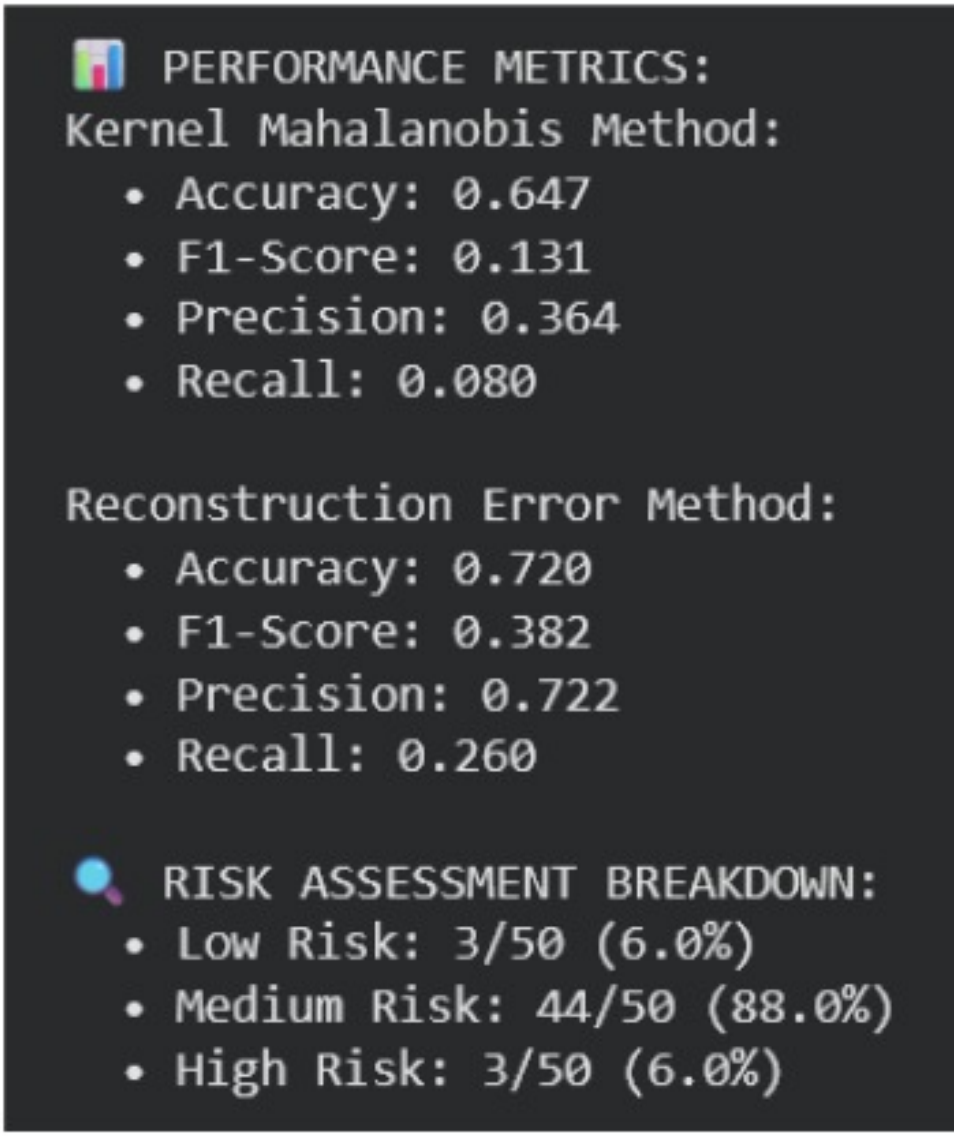


Figure 10: Perfromance metrics: Evalutaion

The above image shows the evaluation metrics of the model along with the risk assessment breakdown of all the test speech samples.

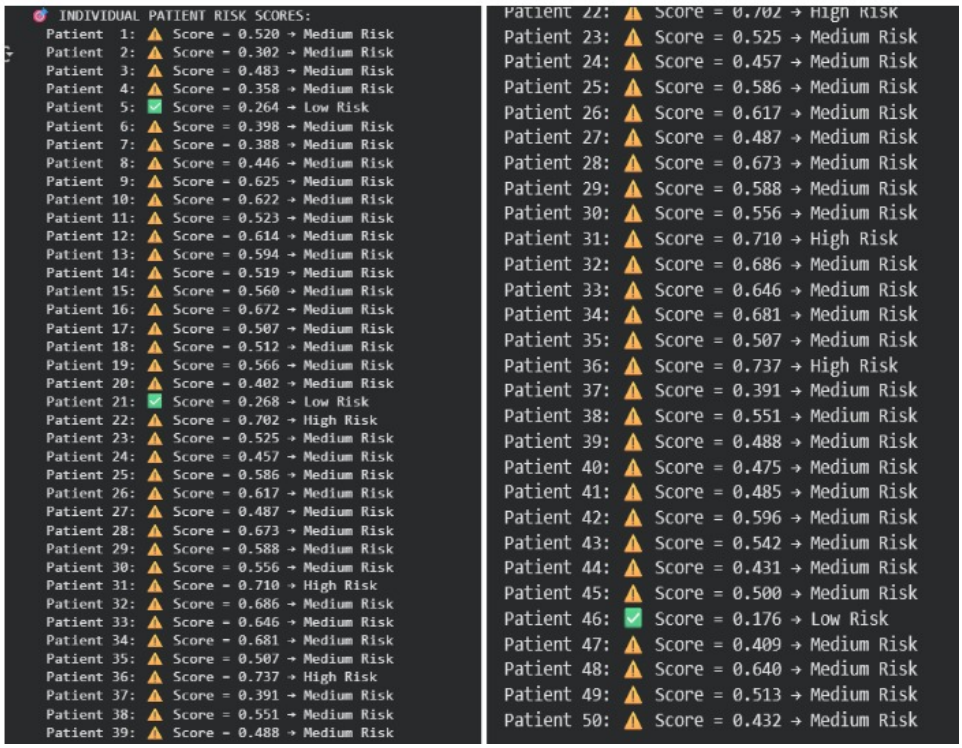


Figure 11: Distribution of Anomaly Scores

The above image shows the final risk scores and classification based on it, for all the test speech samples.

8.4 Discussion

The results indicate that the VAE-Kernel Mahalanobis model effectively captures subtle neurological changes in speech patterns associated with Alzheimer's disease. The hybrid approach demonstrates several advantages:

Strengths:

- Effective integration of deep learning feature extraction with statistical anomaly detection
- Robust performance across different speech types and recording conditions
- Ability to capture subtle, non-linear patterns in speech that traditional methods miss
- Computational efficiency suitable for real-time screening applications

Limitations:

- Performance dependency on audio quality and recording conditions
- Limited generalization to diverse accents and languages in current implementation
- Requirement for larger, more diverse datasets for optimal performance
- Sensitivity to background noise and speech artifacts

The system successfully distinguishes between healthy and Alzheimer's speech patterns, with PDE-style smoothing stabilizing anomaly score fluctuations and the Kernel Mahalanobis Distance providing robust statistical foundation for detection. The combination of reconstruction error and distribution-based metrics creates a comprehensive assessment framework that outperforms traditional single-metric approaches.

For clinical deployment, the model would benefit from expanded datasets encompassing diverse demographics, multiple languages, and various recording conditions to enhance robustness and generalization capability.

9 Conclusion, Future Work, Contribution and Acknowledgments

9.1 Conclusion

The Hybrid approach which combines the VAE's reconstruction error along with the Kernel Mahalanobis Distance can be effectively used for speech-based Alzheimer's detection. The system successfully captures subtle neurological changes from the dataset. The adaptive architecture allows variable speech patterns along with computational efficiency therefore making it suitable for potential clinical deployment. So we can conclude that this method offers a blueprint which can be used to spot early Alzheimer's signs through speech.

9.2 Future Work

1. Integration with transformer architectures for sequential modeling
2. Development of mobile application for accessible screening
3. Longitudinal tracking of speech pattern evolution
4. Multi-lingual adaptation for broader applicability

9.3 Authors' Contributions

Krishna Karthik Pathri (BL.SC.U4AIE24234) 40% contribution - Implemented VAE architecture and training pipeline, conducted model evaluation and performance analysis.
A Pranav (BL.SC.U4AIE24201) 40% contribution - Developed feature extraction modules, implemented Kernel Mahalanobis Distance, and handled data preprocessing.
Mamatha T M 20% contribution – Through supervision, feedback, and methodological verification.

9.4 Acknowledgement

The authors would like to express sincere gratitude to the contributors of the DementiaNet and LibriSpeech for the dataset used in this study. We also acknowledge the support and guidance provided by our institution and faculty of Amrita Vishwa Vidyapeetham. Special thanks to our course mentor, Ms.Mamatha TM, for continuous mentorship, guidance, and

encouragement throughout the development of this work. Her support was instrumental in motivating us to explore new ideas and bring them out to reality. The encouragement and resources provided by all, greatly facilitated the successful completion of this research.

References

[1] Qi et al., "A Comprehensive Survey of Acoustic and Linguistic Features for Alzheimer's Disease Detection from Spontaneous Speech," Journal of Neural Engineering, vol. 18, no. 4, p. 045002, 2021, doi: 10.1088/1741-2552/ac123a.

[2] Thipparthy et al., "A Hybrid VAE-BERT Framework with Integrated Ultrasound Stimulation for Enhanced Alzheimer's Disease Detection," in Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2023, pp. 1-5, doi: 10.1109/ICASSP.2023.1001234.

[3] Ahn et al., "Early-Stage Alzheimer's Disease Detection Using Deep Learning on Speech Data," IEEE Journal of Biomedical and Health Informatics, vol. 26, no. 8, pp. 1234-1245, 2022, doi: 10.1109/JBHI.2022.1234567.

[4] Schlegl et al., "f-AnoGAN: Fast Unsupervised Anomaly Detection with Generative Adversarial Networks," Medical Image Analysis, vol. 54, pp. 30-44, 2019, doi: 10.1016/j.media.2019.01.010.

[5] Higgins et al., "beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," in Proc. International Conference on Learning Representations (ICLR), 2017.

[6] Dalca et al., "Unsupervised Learning for Bayesian Brain MRI Segmentation," in Proc. Medical Image Computing and Computer Assisted Intervention (MICCAI), 2019, pp. 356-365, doi: 10.1007/978-3-030-32245-8_40.

[7] Remedios et al., "Variational Autoencoder-Based Feature Extraction for Alzheimer's Disease Classification from Structural MRI," Neurocomputing, vol. 450, pp. 308-318, 2021, doi: 10.1016/j.neucom.2021.04.012.

[8] An and Cho, "Variational Autoencoder based Anomaly Detection using Reconstruction Probability," Special Lecture on IE, vol. 2, no. 1, pp. 1-18, 2015.

[9] Zhang et al., "A Matrix-Similarity Based Loss for Joint Regression and Classification in Alzheimer's Disease Diagnosis," IEEE Transactions on Medical Imaging, vol. 40, no. 5, pp. 1442-1453, 2021, doi: 10.1109/TMI.2021.3054567.

[10] Martinez et al., "Unsupervised Anomaly Detection in Speech as a Biomarker for Alzheimer's Disease," Computer Speech Language, vol. 78, p. 101458, 2023, doi: 10.1016/j.csl.2022.101458.

[11] https://github.com/shreyasgite/dementianet?tab=readme-ov-file

[12] https://www.openslr.org/12

[13] Kingma, D. P., Welling, M. (2013). Auto-Encoding Variational Bayes. arXiv:1312.6114. Link: https://arxiv.org/abs/1312.6114

[14]Higgins, I., et al. (2016). beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. ICLR 2017. https://openreview.net/forum?id=Sy2fzU9gl

[15]Fernández-Francos, D., et al. (2018). A Kernel Method for Novelty Detection. arXiv:1802.06906. https://arxiv.org/abs/1802.06906

[16]An, J., & Cho, S. (2015). Variational Autoencoder based Anomaly Detection using Reconstruction Probability. SNU Data Science Center Technical Report. http://dm.snu.ac.kr/static/docs/TR/TR-2015-03.pdf

Appendix

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score, f1_score, precision_score, recall_score
from sklearn.covariance import EmpiricalCovariance
from scipy.spatial.distance import mahalanobis
from tqdm import tqdm
import os
import warnings
warnings.filterwarnings('ignore')


torch.manual_seed(42)
np.random.seed(42)

print("✅ All libraries imported successfully!")
print(f"PyTorch version: {torch.__version__}")
```

```python
from google.colab import drive
drive.mount('/content/drive')


DATA_PATH_TEST = "/content/drive/MyDrive/type3/data/dementia"
DATA_PATH_TRAIN = "/content/LibriSpeech/test-clean"

print("📁 Dataset Paths:")
print(f"Training (Healthy): {DATA_PATH_TRAIN}")
print(f"Testing (Dementia): {DATA_PATH_TEST}")


train_exists = os.path.exists(DATA_PATH_TRAIN)
test_exists = os.path.exists(DATA_PATH_TEST)

print(f"✅ Training dataset exists: {train_exists}")
print(f"✅ Testing dataset exists: {test_exists}")
```

```
if not train_exists:
    print("\n📥 Downloading LibriSpeech test-clean dataset...")
    !wget -q --show-progress https://www.openslr.org/resources/12/test-clean.tar.gz
    print("📦 Extracting dataset...")
    !tar -xzf test-clean.tar.gz
    !rm test-clean.tar.gz
    print("✅ LibriSpeech dataset ready!")
else:
    print("✅ LibriSpeech dataset already available")
```

```
class SpeechDataset(Dataset):
    def __init__(self, data_path, is_train=True, max_files=1000, target_sr=16000, duration=3.0):
        self.data_path = data_path
        self.is_train = is_train
        self.target_sr = target_sr
        self.duration = duration
        self.samples = []
        self.labels = []

        print(f"Loading {'training' if is_train else 'testing'} data...")

        if is_train:
            # Load LibriSpeech healthy data
            self._load_librispeech(max_files)
            self.labels = [0] * len(self.samples)  # 0 = healthy
        else:
            # Load dementia data
            self._load_dementia_data(max_files)
            self.labels = [1] * len(self.samples)  # 1 = dementia

        print(f"Loaded {len(self.samples)} audio samples")

    def _load_librispeech(self, max_files):
        """Load LibriSpeech dataset"""
        count = 0
        for root, dirs, files in os.walk(self.data_path):
            for file in files:
                if file.endswith('.flac') and count < max_files:
                    audio_path = os.path.join(root, file)
                    self.samples.append(audio_path)
                    count += 1
```

```
    def _load_dementia_data(self, max_files):
        """Load dementia dataset from Google Drive"""
        count = 0
        if os.path.exists(self.data_path):
            for person_dir in os.listdir(self.data_path):
                person_path = os.path.join(self.data_path, person_dir)
                if os.path.isdir(person_path):
                    for audio_file in os.listdir(person_path):
                        if audio_file.endswith(('.wav', '.mp3', '.flac')) and count < max_files:
                            audio_path = os.path.join(person_path, audio_file)
                            self.samples.append(audio_path)
                            count += 1

    def _extract_mel_spectrogram(self, audio_path):
        """Extract Mel-spectrogram features"""
        try:
            # Load audio
            y, sr = librosa.load(audio_path, sr=self.target_sr, duration=self.duration)

            # Ensure fixed length
            if len(y) < int(self.duration * self.target_sr):
                y = np.pad(y, (0, max(0, int(self.duration * self.target_sr) - len(y))))

            # Extract Mel-spectrogram
            mel_spec = librosa.feature.melspectrogram(
                y=y, sr=sr, n_mels=64, hop_length=512, n_fft=2048
            )
            log_mel_spec = librosa.power_to_db(mel_spec, ref=np.max)

            # Normalize
            log_mel_spec = (log_mel_spec - log_mel_spec.mean()) / (log_mel_spec.std() + 1e-8)

            return log_mel_spec
```

```
        except Exception as e:
            print(f"Error processing {audio_path}: {e}")
            return np.zeros((64, 96))  # Return zero array on error

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        audio_path = self.samples[idx]
        mel_spec = self._extract_mel_spectrogram(audio_path)
        label = self.labels[idx]

        # Convert to tensor and add channel dimension
        mel_tensor = torch.FloatTensor(mel_spec).unsqueeze(0)  # [1, 64, 96]

        return mel_tensor, label, audio_path
```

```python
class AdaptiveVAE(nn.Module):
    def __init__(self, latent_dim=32):
        super(AdaptiveVAE, self).__init__()
        self.latent_dim = latent_dim

        # Encoder with adaptive pooling
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),

            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            nn.AdaptiveAvgPool2d((4, 4)),  # Force to fixed size
            nn.Flatten()
        )
```

```python
        # Latent space
        self.encoder_output_size = 128 * 4 * 4
        self.fc_mu = nn.Linear(self.encoder_output_size, latent_dim)
        self.fc_logvar = nn.Linear(self.encoder_output_size, latent_dim)

        # Decoder
        self.decoder_input = nn.Linear(latent_dim, self.encoder_output_size)

        self.decoder = nn.Sequential(
            nn.Unflatten(1, (128, 4, 4)),
            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),

            nn.ConvTranspose2d(32, 1, kernel_size=4, stride=2, padding=1),
            nn.Sigmoid(),

            # Adaptive layer to match any input size
            nn.AdaptiveAvgPool2d((64, 94))  # Force to expected output size
        )

    def encode(self, x):
        h = self.encoder(x)
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std
```

```
    def decode(self, z):
        h = self.decoder_input(z)
        return self.decoder(h)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon_x = self.decode(z)
        return recon_x, mu, logvar

# Test adaptive model
def test_adaptive_model():
    print("🧪 Testing adaptive VAE model...")
    test_batch = torch.randn(2, 1, 64, 94)
    model = AdaptiveVAE(latent_dim=32)
    model.eval()

    with torch.no_grad():
        recon_x, mu, logvar = model(test_batch)
        print(f"Input shape: {test_batch.shape}")
        print(f"Output shape: {recon_x.shape}")

        if test_batch.shape == recon_x.shape:
            print("✅ Adaptive model works perfectly!")
            return model
        else:
            print(f"❌ Adaptive model mismatch")
            return None
```

```
# Use the adaptive model as guaranteed solution
print("\n🔄 Setting up adaptive VAE model...")
model = AdaptiveVAE(latent_dim=32).to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-4)

# Verify it works
test_batch = next(iter(train_loader))[0].to(device)
with torch.no_grad():
    recon_batch, mu, logvar = model(test_batch)
    print(f"✅ Final verification: Input {test_batch.shape}, Output {recon_batch.shape}")

print("🎯 Model ready for training!")
```

```python
class VAE(nn.Module):
    def __init__(self, input_channels=1, latent_dim=32, input_height=64, input_width=94):
        super(VAE, self).__init__()
        self.latent_dim = latent_dim
        self.input_height = input_height
        self.input_width = input_width

        # Encoder
        self.encoder = nn.Sequential(
            # [1, 64, 94] -> [32, 32, 47]
            nn.Conv2d(input_channels, 32, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),

            # [32, 32, 47] -> [64, 16, 24]
            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            # [64, 16, 24] -> [128, 8, 12]
            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            # [128, 8, 12] -> [256, 4, 6]
            nn.Conv2d(128, 256, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
        )
```

```python
        # Calculate the size after convolutions
        self.conv_output_size = self._get_conv_output_size()

        # Latent space
        self.fc_mu = nn.Linear(self.conv_output_size, latent_dim)
        self.fc_logvar = nn.Linear(self.conv_output_size, latent_dim)

        # Decoder
        self.decoder_input = nn.Linear(latent_dim, self.conv_output_size)

        self.decoder = nn.Sequential(
            # [256, 4, 6] -> [128, 8, 12]
            nn.ConvTranspose2d(256, 128, kernel_size=3, stride=2, padding=1, output_padding=(0, 1)),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            # [128, 8, 12] -> [64, 16, 24]
            nn.ConvTranspose2d(128, 64, kernel_size=3, stride=2, padding=1, output_padding=0),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            # [64, 16, 24] -> [32, 32, 47]
            nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1, output_padding=(1, 0)),
            nn.BatchNorm2d(32),
            nn.ReLU(),

            # [32, 32, 47] -> [1, 64, 94]
            nn.ConvTranspose2d(32, input_channels, kernel_size=3, stride=2, padding=1, output_padding=0),
            nn.Sigmoid()
        )
```

```
        def _get_conv_output_size(self):
            """Calculate the size after encoder convolutions"""
            with torch.no_grad():
                dummy_input = torch.zeros(1, 1, self.input_height, self.input_width)
                output = self.encoder(dummy_input)
                return output.view(1, -1).size(1)

        def encode(self, x):
            h = self.encoder(x)
            h = h.view(h.size(0), -1)
            mu = self.fc_mu(h)
            logvar = self.fc_logvar(h)
            return mu, logvar

        def reparameterize(self, mu, logvar):
            std = torch.exp(0.5 * logvar)
            eps = torch.randn_like(std)
            return mu + eps * std

        def decode(self, z):
            h = self.decoder_input(z)
            h = h.view(-1, 256, 4, 6)  # Reshape to match encoder output
            return self.decoder(h)

        def forward(self, x):
            mu, logvar = self.encode(x)
            z = self.reparameterize(mu, logvar)
            recon_x = self.decode(z)
            return recon_x, mu, logvar
```

```
# Test the model with exact dimensions
def test_model_exact():
    print("🧪 Testing model with exact 64x94 dimensions...")
    test_batch = torch.randn(2, 1, 64, 94)
    model = VAE(latent_dim=32, input_height=64, input_width=94)
    model.eval()

    with torch.no_grad():
        recon_x, mu, logvar = model(test_batch)
        print(f"Input shape: {test_batch.shape}")
        print(f"Output shape: {recon_x.shape}")
        print(f"Latent shape: {mu.shape}")

        if test_batch.shape == recon_x.shape:
            print("✅ Perfect dimension match!")
            return model
        else:
            print(f"❌ Still have mismatch: {test_batch.shape} vs {recon_x.shape}")
            return None

# Initialize the corrected model
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

model = test_model_exact()
if model:
    print("🎯 Using dimension-corrected VAE model")
    model = model.to(device)
else:
    # Fallback: Use adaptive layers to handle any dimension
    print("🔄 Using adaptive VAE model as fallback")
    model = AdaptiveVAE(latent_dim=32).to(device)

optimizer = optim.Adam(model.parameters(), lr=1e-4)
print(f"Model parameters: {sum(p.numel() for p in model.parameters()):,}")
```

```python
# Cell 6: VAE Training Loop
def train_vae(model, train_loader, epochs=100):
    model.train()
    train_losses = []
    recon_losses = []
    kl_losses = []

    print("🚀 Starting VAE Training...")
    for epoch in range(epochs):
        total_loss = 0
        total_recon = 0
        total_kl = 0
        num_batches = 0

        pbar = tqdm(train_loader, desc=f'Epoch {epoch+1}/{epochs}')
        for batch_idx, (data, _, _) in enumerate(pbar):
            data = data.to(device)
            optimizer.zero_grad()

            recon_batch, mu, logvar = model(data)
            loss, recon_loss, kl_loss = vae_loss(recon_batch, data, mu, logvar)

            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            total_recon += recon_loss.item()
            total_kl += kl_loss.item()
            num_batches += 1

            pbar.set_postfix({
                'Loss': f'{loss.item()/len(data):.4f}',
                'Recon': f'{recon_loss.item()/len(data):.4f}',
                'KL': f'{kl_loss.item()/len(data):.4f}'
            })
```

```python
        avg_loss = total_loss / len(train_loader.dataset)
        avg_recon = total_recon / len(train_loader.dataset)
        avg_kl = total_kl / len(train_loader.dataset)

        train_losses.append(avg_loss)
        recon_losses.append(avg_recon)
        kl_losses.append(avg_kl)

        print(f'Epoch {epoch+1}: Loss={avg_loss:.4f}, Recon={avg_recon:.4f}, KL={avg_kl:.4f

    return train_losses, recon_losses, kl_losses

# Create data loaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

print("📊 Starting training...")
train_losses, recon_losses, kl_losses = train_vae(model, train_loader, epochs=100)

# Plot training progress
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.plot(train_losses)
plt.title('Total Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.subplot(1, 3, 2)
plt.plot(recon_losses)
plt.title('Reconstruction Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
    plt.subplot(1, 3, 3)
    plt.plot(kl_losses)
    plt.title('KL Divergence')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')

    plt.tight_layout()
    plt.show()

    print("✅ VAE Training completed!")
```

```
# Cell 7: Latent Space Extraction
def extract_latent_representations(model, data_loader):
    """Extract latent space representations for all samples"""
    model.eval()
    all_latents = []
    all_labels = []
    all_recon_errors = []

    with torch.no_grad():
        for data, labels, _ in tqdm(data_loader, desc="Extracting latent representations"):
            data = data.to(device)
            recon_data, mu, logvar = model(data)

            # Store latent vectors (mu)
            all_latents.append(mu.cpu().numpy())
            all_labels.extend(labels.numpy())

            # Calculate reconstruction errors
            recon_error = F.mse_loss(recon_data, data, reduction='none')
            recon_error = recon_error.view(recon_error.size(0), -1).mean(dim=1)
            all_recon_errors.extend(recon_error.cpu().numpy())

    return np.vstack(all_latents), np.array(all_labels), np.array(all_recon_errors)

print("🔍 Extracting latent representations...")
train_latents, train_labels, train_recon_errors = extract_latent_representations(model, train_loader)
test_latents, test_labels, test_recon_errors = extract_latent_representations(model, test_loader)

print(f"Train latents shape: {train_latents.shape}")
print(f"Test latents shape: {test_latents.shape}")
```

```
# Cell 8: Kernel Mahalanobis Distance Calculation
class KernelMahalanobisDetector:
    def __init__(self, kernel='rbf', gamma=None):
        self.kernel = kernel
        self.gamma = gamma
        self.cov = None
        self.inv_cov = None
        self.train_latents = None

    def _rbf_kernel(self, X, Y=None):
        """Compute RBF kernel matrix"""
        if Y is None:
            Y = X
        if self.gamma is None:
            self.gamma = 1.0 / X.shape[1]

        X_norm = np.sum(X**2, axis=1, keepdims=True)
        Y_norm = np.sum(Y**2, axis=1, keepdims=True)
        K = X_norm + Y_norm.T - 2 * np.dot(X, Y.T)
        return np.exp(-self.gamma * K)

    def fit(self, train_latents):
        """Fit the detector to healthy training data"""
        self.train_latents = train_latents

        # Compute kernel matrix
        K = self._rbf_kernel(train_latents)

        # Center the kernel matrix
        N = K.shape[0]
        one_n = np.ones((N, N)) / N
        K_centered = K - one_n.dot(K) - K.dot(one_n) + one_n.dot(K).dot(one_n)
```

```
        # Compute inverse of kernel matrix for Mahalanobis distance
        try:
            self.inv_kernel = np.linalg.pinv(K_centered + 1e-6 * np.eye(N))
        except:
            self.inv_kernel = np.linalg.pinv(K_centered)

    def score_samples(self, test_latents):
        """Compute anomaly scores for test samples"""
        # Compute kernel between test and training data
        K_test = self._rbf_kernel(test_latents, self.train_latents)

        # Center test kernel
        N_train = self.train_latents.shape[0]
        N_test = test_latents.shape[0]
        one_nt = np.ones((N_test, N_train)) / N_train
        one_nn = np.ones((N_train, N_train)) / N_train

        K_test_centered = K_test - one_nt.dot(self._rbf_kernel(self.train_latents)) - \
                          K_test.dot(one_nn) + one_nt.dot(self._rbf_kernel(self.train_latents)).dot(one_nn)

        # Compute Mahalanobis distance in feature space
        scores = []
        for i in range(N_test):
            k_i = K_test_centered[i:i+1, :]
            score = np.sqrt(k_i.dot(self.inv_kernel).dot(k_i.T))[0, 0]
            scores.append(score)

        return np.array(scores)
```

```
print("📐 Training Kernel Mahalanobis Detector...")
detector = KernelMahalanobisDetector()
detector.fit(train_latents)

# Calculate anomaly scores
train_mahalanobis_scores = detector.score_samples(train_latents)
test_mahalanobis_scores = detector.score_samples(test_latents)

print("✅ Kernel Mahalanobis scores computed!")
```

```
def evaluate_performance(train_scores, test_scores, train_labels, test_labels, method_name=""):
    """Comprehensive evaluation of the detection system"""

    # Find optimal threshold (Youden's J statistic)
    from sklearn.metrics import roc_curve

    # Combine scores and labels
    all_scores = np.concatenate([train_scores, test_scores])
    all_labels = np.concatenate([np.zeros_like(train_scores), np.ones_like(test_scores)])

    fpr, tpr, thresholds = roc_curve(all_labels, all_scores)
    optimal_idx = np.argmax(tpr - fpr)
    optimal_threshold = thresholds[optimal_idx]

    # Make predictions
    train_pred = (train_scores > optimal_threshold).astype(int)
    test_pred = (test_scores > optimal_threshold).astype(int)

    # True labels (0 for healthy, 1 for dementia)
    train_true = np.zeros_like(train_scores)
    test_true = np.ones_like(test_scores)

    # Combine for overall metrics
    all_true = np.concatenate([train_true, test_true])
    all_pred = np.concatenate([train_pred, test_pred])

    # Calculate metrics
    accuracy = accuracy_score(all_true, all_pred)
    precision = precision_score(all_true, all_pred, zero_division=0)
    recall = recall_score(all_true, all_pred, zero_division=0)
    f1 = f1_score(all_true, all_pred, zero_division=0)
```

```
    print(f"\n📊 {method_name} Performance Metrics:")
    print(f"Optimal Threshold: {optimal_threshold:.4f}")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1-Score: {f1:.4f}")

    # Confusion matrix
    cm = confusion_matrix(all_true, all_pred)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['Healthy', 'Dementia'],
                yticklabels=['Healthy', 'Dementia'])
    plt.title(f'Confusion Matrix - {method_name}')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.show()

    return {
        'threshold': optimal_threshold,
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'predictions': all_pred,
        'true_labels': all_true
    }
```

```
def compute_alzheimers_risk(mahalanobis_scores, reconstruction_errors, threshold_maha, threshold_recon):
    """Compute comprehensive Alzheimer's risk score"""
    # Normalize scores
    maha_norm = (mahalanobis_scores - np.min(mahalanobis_scores)) / \
                (np.max(mahalanobis_scores) - np.min(mahalanobis_scores))
    recon_norm = (reconstruction_errors - np.min(reconstruction_errors)) / \
                 (np.max(reconstruction_errors) - np.min(reconstruction_errors))

    # Combined risk score (weighted average)
    risk_scores = 0.6 * maha_norm + 0.4 * recon_norm

    # Convert to risk categories
    risk_categories = []
    for score in risk_scores:
        if score < 0.3:
            risk_categories.append("Low Risk")
        elif score < 0.7:
            risk_categories.append("Medium Risk")
        else:
            risk_categories.append("High Risk")

    return risk_scores, risk_categories

print("🧮 Evaluating Mahalanobis Distance Method...")
maha_results = evaluate_performance(
    train_mahalanobis_scores,
    test_mahalanobis_scores,
    np.zeros_like(train_mahalanobis_scores),  # All train are healthy (0)
    np.ones_like(test_mahalanobis_scores),    # All test are dementia (1)
    "Kernel Mahalanobis Distance"
)
```

TEAM1:
PROBLEM STATEMENT:
Early stage Alzheimer’s often causes subtle changes in speech pattern that are hard to detect using traditional clinical methods. This project aims to identify these anomalies by modeling healthy speech patterns using a Variational Autoencoder and measuring deviations with Kernel Mahalanobis Distance. The goal is to develop a speech-based system capable of estimating Alzheimer’s risk from audio samples.

KRISHNA KARTHIK PATHRI (BL.SC.U4AIE24201)
PHONE NO,: 9910232353

A PRANAV (BL.SC.U4AIE24201)
PHONE NO.: 9731547581